

```

// RTL description of design example (see Fig. 8.11)
module Design_Example_RTL (A, E, F, Start, clock, reset_b);
    // Specify ports of the top-level module of the design
    // See block diagram, Fig. 8.10
    output [3: 0] A;
    output    E, F;
    input     Start, clock, reset_b;

    // Instantiate controller and datapath units
    Controller_RTL M0 (set_E, clr_E, set_F, clr_A_F, incr_A, A[2], A[3], Start, clock,
reset_b);
    Datapath_RTL M1 (A, E, F, set_E, clr_E, set_F, clr_A_F, incr_A, clock);
endmodule

module Controller_RTL (set_E, clr_E, set_F, clr_A_F, incr_A, A2, A3, Start, clock,
reset_b);
    output reg    set_E, clr_E, set_F, clr_A_F, incr_A;
    input    Start, A2, A3, clock, reset_b;
    reg [1: 0]    state, next_state;
    parameter    S_idle = 2'b00, S_1 = 2'b01, S_2 = 2'b11;      // State codes
    always @ (posedge clock, negedge reset_b) // State transitions (edge
sensitive)
        if (reset_b == 0) state <= S_idle;
        else state <= next_state;
    // Code next-state logic directly from ASMD chart (Fig. 8.9d)
    always @ (state, Start, A2, A3) begin // Next-state logic (level sensitive)
        next_state = S_idle;
        case (state)
            S_idle: if (Start) next_state = S_1; else next_state = S_idle;
            S_1:    if (A2 & A3) next_state = S_2; else next_state = S_1;
            S_2:    next_state = S_idle;
            default:    next_state = S_idle;
        endcase
    end
    // Code output logic directly from ASMD chart (Fig. 8.9d)
    always @ (state, Start, A2) begin
        set_E    = 0; // default assignments; assign by exception
        clr_E    = 0;
        set_F    = 0;
        clr_A_F = 0;
        incr_A   = 0;
        case (state)
            S_idle: if (Start) clr_A_F = 1;
            S_1:    begin incr_A = 1; if (A2) set_E = 1; else clr_E = 1; end
            S_2:    set_F = 1;
        endcase
    end
endmodule

```

```

module Datapath_RTL (A, E, F, set_E, clr_E, set_F, clr_A_F, incr_A, clock);
  output reg [3: 0] A;           // register for counter
  output reg          E, F;     // flags
  input    set_E, clr_E, set_F, clr_A_F, incr_A, clock;
  // Code register transfer operations directly from ASMD chart (Fig. 8.9(d))
  always @ (posedge clock) begin
    if (set_E)           E <= 1;
    if (clr_E)           E <= 0;
    if (set_F)           F <= 1;
    if (clr_A_F)         begin A <= 0; F <= 0; end
    if (incr_A)          A <= A + 1;
  end
endmodule

```
